EinsteinKPy2Tex: A Python Tool for Computing and Exporting General Relativity Tensors to LaTeX

Pierros Ntelis[†,‡]

[†] *School of Physics, Harbin Institute of Technology, Harbin 150001, People's Republic of China*
[‡] *Institute of Theoretical Physics, National University of Uzbekistan, Tashkent 100174, Uzbekistan*

June 11, 2026; conceived: 5 June 2026

Abstract

We present EinsteinKPy2Tex, an open-source Python package that computes fundamental geometric quantities for the Friedmann-Lemaître-Robertson-Walker (FLRW) and Schwarzschild metrics using the EinsteinPy library. The code automatically calculates Christoffel symbols, Riemann curvature tensor, Ricci tensor, Ricci scalar, Einstein tensor, geodesic equations, and Kretschmann scalar. Results are exported as professionally formatted LaTeX documents, facilitating direct inclusion in research papers and educational materials.

The FLRW implementation is essential for cosmology, enabling the study of the universe's large-scale structure and dynamics. The Schwarzschild implementation is fundamental for black hole physics, describing the spacetime geometry around non-rotating compact objects. The code includes built-in citations for the original Einstein paper (1915) and the EinsteinPy library (Ribeiro et al. 2020), ensuring proper academic attribution.

In addition, the code computes the Ricci squared invariant $R_{\mu\nu}R^{\mu\nu}$, the Gauss–Bonnet scalar $\mathcal{R}_{\rm GB} = R^2 - 4R_{\mu\nu}R^{\mu\nu} + R_{\mu\nu\rho\sigma}R^{\mu\nu\rho\sigma}$, the Weyl tensor $C^{\mu}_{\nu\rho\sigma}$, and the Weyl squared invariant $C_{\mu\nu\rho\sigma}C^{\mu\nu\rho\sigma}$, providing a complete set of quadratic curvature invariants.

EinsteinKPy2Tex is designed for educators, students, and researchers needing rapid, accurate tensor calculations with publication-ready output. The software is available at GitHub under an MIT license.

Keywords: General Relativity ; Schwarzschild metric ; FLRW metric ; Kretschmann scalar ; Geodesic equations ; Tensor calculus ; LaTeX export ; Symbolic computation ; EinsteinPy ; EinsteinKPy2Tex ; Cosmology ; Black hole physics

Contents

1 Introduction

General relativity (GR) remains the cornerstone of modern astrophysics, from cosmological models of the expanding Universe to the description of black hole spacetimes. However, calculating the fundamental tensors—Christoffel symbols, the Riemann curvature tensor, the Ricci tensor and scalar, and the Einstein tensor—for even moderately complex metrics is error-prone and time-consuming when performed manually. While symbolic computation packages like SymPy [1] and specialized libraries like EinsteinPy [2] exist, there remains a gap between symbolic calculation and publication-ready output. Researchers often need to verify intermediate steps, derive geodesic equations, or compute curvature invariants such as the Kretschmann scalar, all while maintaining a clear record of their calculations for papers and theses.

This need is particularly acute in cosmology and black hole astrophysics, where sophisticated software infrastructure has become essential for modern research. In cosmology, the community has developed numerous public codes for computing cosmic microwave background (CMB) anisotropies and large-scale structure. Notable examples include `CAMB` (Code for Anisotropies in the Microwave Background) [3], `CLASS` (Cosmic Linear Anisotropy Solving System) [4, 5, 6], and the earlier `CMBFAST` [7]. These Boltzmann codes solve the full Einstein-Boltzmann hierarchy and have enabled precision cosmology, from the WMAP era to Planck [8]. They are often used in conjunction with

parameter estimation frameworks such as `CosmoMC` [9] and `MontePython` [10], and more recently with neural network-based emulators like `CosmoPower` [11] that accelerate cosmological inference.

In parallel, the black hole and gravitational wave communities have developed their own specialized software for computing geodesics, orbital trajectories, and gravitational waveforms. While these existing codes are powerful, they are typically designed for large-scale numerical computations, not for educational purposes or for generating clean, publication-ready symbolic output.

`EinsteinKPy2Tex` fills this niche. It is a lightweight Python tool that bridges the gap between symbolic tensor calculations and LaTeX export, specifically designed for the FLRW and Schwarzschild metrics. The code:

- Computes all standard GR tensors (metric, Christoffel, Riemann, Ricci, Einstein)
- Derives the geodesic equations automatically from the Christoffel symbols
- Calculates the Kretschmann scalar $K = R^{\mu\nu\rho\sigma}R_{\mu\nu\rho\sigma}$, a key curvature invariant for identifying singularities
- Computes the Ricci squared invariant $R_{\mu\nu}R^{\mu\nu}$ and the Gauss–Bonnet scalar $\mathcal{R}_{\rm GB} = R^2 - 4R_{\mu\nu}R^{\mu\nu} + R_{\mu\nu\rho\sigma}R^{\mu\nu\rho\sigma}$
- Evaluates the Weyl tensor $C^{\mu}_{\nu\rho\sigma}$ and the Weyl squared invariant $C_{\mu\nu\rho\sigma}C^{\mu\nu\rho\sigma}$
- Exports all non-zero components as ready-to-compile LaTeX documents with proper index spacing
- Includes built-in citations for Einstein (1915), the EinsteinPy library [2], and this software

The FLRW implementation enables studies of cosmological dynamics, including the dependence of curvature invariants on the scale factor $a(t)$ and its derivatives. The Schwarzschild implementation provides the foundation for black hole physics, correctly recovering the vacuum solution $R_{\mu\nu} = 0$ and the Kretschmann scalar $K = 48G^2M^2/(c^4r^6)$, which signals the physical singularity at $r = 0$.

`EinsteinKPy2Tex` is designed for educators, students, and researchers needing rapid, accurate tensor calculations with publication-ready output. Under an MIT license, the software is available, at `https://github.com/lontelis/EinsteinKPy2Tex`.

2 Software Architecture

Figure 1 presents a flowchart illustrating the algorithm implemented in both scripts.

After starting the code, we first define the important quantities, i.e., the coordinates and symbols. We then define the metric components. These definitions are described within the gray area.

Next, we perform tensor computations via EinsteinPy, followed by the computation of the geodesic equations using symbolic Python code. We then compute a full set of quadratic curvature invariants: the Kretschmann scalar $K = R^{\mu\nu\rho\sigma}R_{\mu\nu\rho\sigma}$, the Ricci squared invariant $R_{\mu\nu}R^{\mu\nu}$, the Gauss–Bonnet scalar $\mathcal{R}_{\rm GB} = R^2 - 4R_{\mu\nu}R^{\mu\nu} + R_{\mu\nu\rho\sigma}R^{\mu\nu\rho\sigma}$, and the Weyl squared invariant $C_{\mu\nu\rho\sigma}C^{\mu\nu\rho\sigma}$. The Weyl tensor itself is also computed. All invariant calculations are performed symbolically, using index raising and lowering with the metric.

The tensor computations are represented with green colors in the flowchart. Finally, using a Python-LaTeX interface, we translate the computations into LaTeX format, represented with pink color. The generated document includes only non-zero components, with proper spacing for multi-character indices.

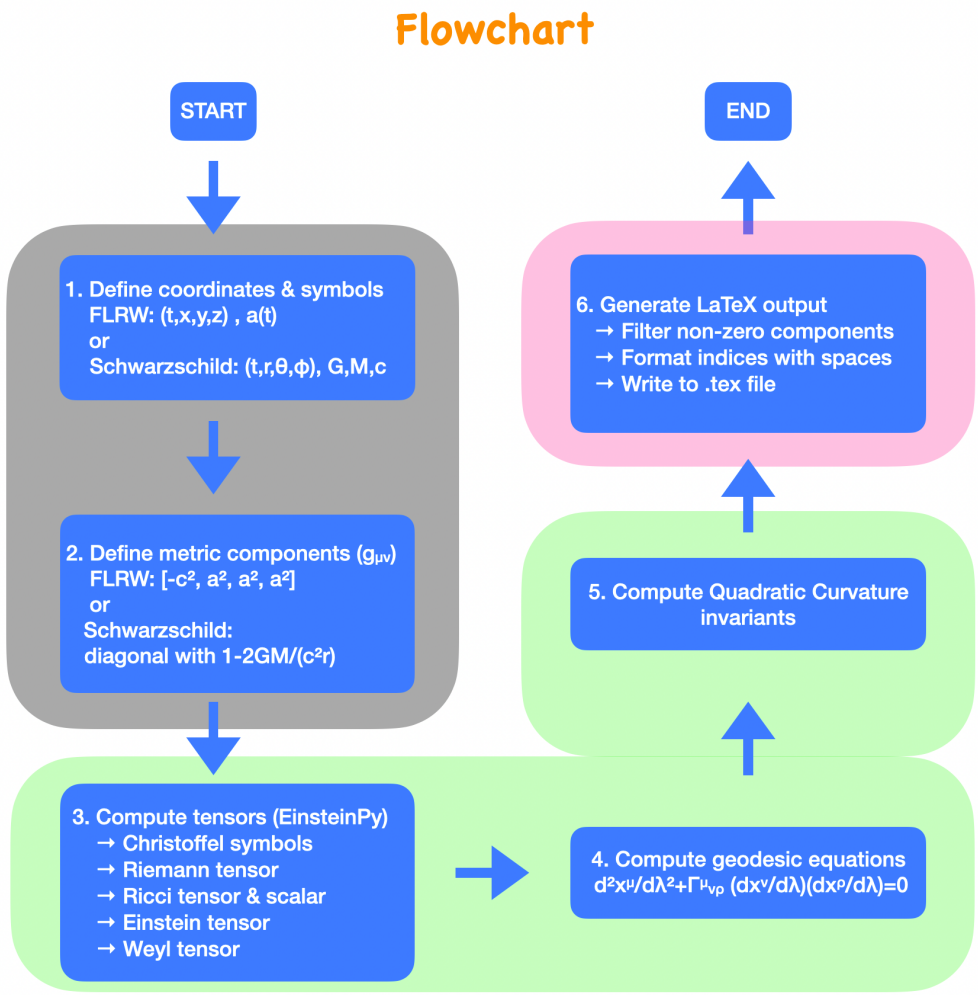

Figure 1: Flowchart of `EinsteinKPy2Tex`. The code follows a linear pipeline: metric definition, tensor computation via EinsteinPy, derivation of geodesic equations, calculation of quadratic curvature invariants (Kretschmann, Ricci squared, Gauss–Bonnet, Weyl squared), and LaTeX export.

2.1 Dependencies

The software requires:

- Python 3.7 or higher
- SymPy [1] for symbolic mathematics
- NumPy [12] for numerical array operations
- EinsteinPy [2] for GR tensor operations

Installation is straightforward via pip:

```
pip install sympy numpy einsteinpy
```

2.2 Code Structure

The package consists of two independent scripts:

- `flrw_EinsteinKPy2Tex.py`: FLRW cosmological metric with scale factor $a(t)$
- `schw_EinsteinKPy2Tex.py`: Static spherically symmetric metric

Each script follows a modular design:

1. Metric definition in Cartesian or spherical coordinates
2. Tensor computation via EinsteinPy API calls
3. Geodesic equation derivation
4. Kretschmann scalar calculation
5. LaTeX output generation with automatic spacing for multi-character indices

3 Physical Quantities Computed

The software computes the following quantities for both metrics:

3.1 Metric Tensor $g_{\mu\nu}$

The fundamental geometric quantity defining spacetime intervals:

$$ds^2 = g_{\mu\nu} dx^\mu dx^\nu \tag{1}$$

3.2 Christoffel Symbols $\Gamma^\mu_{\nu\rho}$

The connection coefficients encoding the metric's affine structure:

$$\Gamma^\mu_{\nu\rho} = \frac{1}{2} g^{\mu\sigma} \left(\partial_\nu g_{\sigma\rho} + \partial_\rho g_{\sigma\nu} - \partial_\sigma g_{\nu\rho} \right) \tag{2}$$

3.3 Riemann Curvature Tensor $R^\mu_{\nu\rho\sigma}$

Measures the intrinsic curvature of spacetime:

$$R^\mu_{\nu\rho\sigma} = \partial_\rho \Gamma^\mu_{\nu\sigma} - \partial_\sigma \Gamma^\mu_{\nu\rho} + \Gamma^\mu_{\rho\lambda} \Gamma^\lambda_{\nu\sigma} - \Gamma^\mu_{\sigma\lambda} \Gamma^\lambda_{\nu\rho} \tag{3}$$

3.4 Ricci Tensor $R_{\mu\nu}$ and Scalar R

The contractions of the Riemann tensor:

$$R_{\mu\nu} = R^\lambda_{\mu\lambda\nu}, \quad R = g^{\mu\nu} R_{\mu\nu} \tag{4}$$

3.5 Einstein Tensor $G_{\mu\nu}$

The curvature-matter coupling in GR:

$$G_{\mu\nu} = R_{\mu\nu} - \frac{1}{2} R g_{\mu\nu} \tag{5}$$

3.6 Geodesic Equations

The equations governing free-fall motion:

$$\frac{d^2 x^\mu}{d\lambda^2} + \Gamma^\mu_{\nu\rho} \frac{dx^\nu}{d\lambda} \frac{dx^\rho}{d\lambda} = 0 \tag{6}$$

3.7 Kretschmann Scalar K

A quadratic curvature invariant useful for identifying singularities:

$$K = R^{\mu\nu\rho\sigma} R_{\mu\nu\rho\sigma} \tag{7}$$

3.8 Ricci Squared $R_{\mu\nu}R^{\mu\nu}$

A quadratic curvature invariant constructed from the Ricci tensor:

$$R_{\mu\nu}R^{\mu\nu} = g^{\mu\alpha}g^{\nu\beta}R_{\mu\nu}R_{\alpha\beta}. \tag{8}$$

This invariant provides additional information about the curvature beyond the Ricci scalar and is non-zero for many spacetimes that have $R = 0$ (e.g., the Schwarzschild metric). In vacuum, it vanishes identically, while in the presence of matter it is determined by the Einstein equations.

3.9 Gauss–Bonnet Scalar $\mathcal{R}_{\mathrm{GB}}$

In four dimensions, the Gauss–Bonnet scalar is a topological invariant (up to a boundary term) and is given by:

$$\mathcal{R}_{\mathrm{GB}} = R^2 - 4R_{\mu\nu}R^{\mu\nu} + R_{\mu\nu\rho\sigma}R^{\mu\nu\rho\sigma}. \tag{9}$$

It plays an important role in modified theories of gravity and in the Euclidean path integral approach to quantum gravity.

3.10 Weyl Tensor $C^{\mu}_{\ \nu\rho\sigma}$ and Weyl Squared $C_{\mu\nu\rho\sigma}C^{\mu\nu\rho\sigma}$

The Weyl tensor represents the traceless part of the Riemann tensor and encodes purely tidal gravitational fields. It is defined in four dimensions as:

$$C_{\mu\nu\rho\sigma} = R_{\mu\nu\rho\sigma} - \frac{1}{2}\left(g_{\mu\rho}R_{\nu\sigma} - g_{\mu\sigma}R_{\nu\rho} - g_{\nu\rho}R_{\mu\sigma} + g_{\nu\sigma}R_{\mu\rho}\right) + \frac{R}{6}\left(g_{\mu\rho}g_{\nu\sigma} - g_{\mu\sigma}g_{\nu\rho}\right). \tag{10}$$

The Weyl squared invariant $C_{\mu\nu\rho\sigma}C^{\mu\nu\rho\sigma}$ measures the strength of the tidal field and is non-zero for gravitational waves and for vacuum spacetimes that are not conformally flat. For conformally flat spacetimes such as the FLRW universe, the Weyl tensor vanishes identically.

4 Implementation Details

4.1 FLRW Metric

The Friedmann-Lemaître-Robertson-Walker metric for a flat universe is:

$$ds^2 = -c^2dt^2 + a(t)^2(dx^2 + dy^2 + dz^2) \tag{11}$$

where $a(t)$ is the scale factor and c is the speed of light. This metric describes a homogeneous and isotropic expanding universe.

4.2 Schwarzschild Metric

The Schwarzschild metric for a non-rotating black hole is:

$$ds^2 = -\left(1 - \frac{2GM}{c^2r}\right)c^2dt^2 + \left(1 - \frac{2GM}{c^2r}\right)^{-1}dr^2 + r^2d\theta^2 + r^2\sin^2\theta d\phi^2 \tag{12}$$

where G is Newton's gravitational constant and M is the black hole mass.

4.3 Geodesic Implementation

The geodesic equations are derived from:

$$\ddot{x}^{\mu} + \Gamma^{\mu}_{\nu\rho}\dot{x}^{\nu}\dot{x}^{\rho} = 0 \tag{13}$$

where dots denote derivatives with respect to the affine parameter λ. The code computes symbolic expressions for each coordinate.

4.4 Quadratic Curvature Invariants Implementation

All quadratic invariants are computed by the same general method: lower the first index of the corresponding tensor (Riemann, Ricci, or Weyl) to obtain a fully covariant tensor, raise all indices using the inverse metric, and contract. The code implements this procedure for:

- **Kretschmann scalar**: $K = R_{\mu\nu\rho\sigma}R^{\mu\nu\rho\sigma}$.
- **Ricci squared**: $R_{\mu\nu}R^{\mu\nu}$.
- **Weyl squared**: $C_{\mu\nu\rho\sigma}C^{\mu\nu\rho\sigma}$.

The Gauss–Bonnet scalar is then constructed from the Ricci scalar, Ricci squared, and Kretschmann scalar as $\mathcal{R}_{\mathrm{GB}} = R^2 - 4R_{\mu\nu}R^{\mu\nu} + R_{\mu\nu\rho\sigma}R^{\mu\nu\rho\sigma}$.

For the Schwarzschild metric, the code correctly returns $R_{\mu\nu} = 0$ and thus $R_{\mu\nu}R^{\mu\nu} = 0$, $\mathcal{R}_{\mathrm{GB}} = K$. For the FLRW metric, the Weyl tensor vanishes, yielding $C_{\mu\nu\rho\sigma}C^{\mu\nu\rho\sigma} = 0$, which serves as an additional validation of the implementation.

5 Usage Examples

5.1 Basic Execution

To compute tensors for the FLRW metric:

```
python flrw_EinsteinKPy2Tex.py
```

For the Schwarzschild metric:

```
python schw_EinsteinKPy2Tex.py
```

5.2 Citation Management

Display citation information:

```
python einsteinpy_to_latex.py --cite
```

Generate BibTeX citation file:

```
python flrw_EinsteinKPy2Tex.py --generate_bib
```

5.3 LaTeX Compilation

Compile the generated LaTeX document to PDF:

```
pdflatex flrw_EinsteinKPy2Tex.tex
```

6 Customizing for Your Own Metric

The software is designed to be easily adaptable to any metric beyond the built-in FLRW and Schwarzschild cases. Users can compute tensors for custom spacetimes by modifying only two sections of the code.

6.1 Modification Steps

To compute tensors for a custom metric, the user modifies the following sections in the script:

1. Define coordinates and functions

```
# 1. Coordinates and functions
t, x, y, z = sp.symbols("t␣x␣y␣z")
coords = [t, x, y, z]
coord_names = ['t', 'x', 'y', 'z']  # for LaTeX labels

a = sp.Function("a")(t)  # Example: scale factor
c = sp.Symbol("c", positive=True, real=True)  # Speed of light
```

2. Define metric components

```
# 2. Metric (4D)
metric_arr = [
    [-c**2, 0, 0, 0],       # g_tt component
    [0, a**2, 0, 0],        # g_xx component
    [0, 0, a**2, 0],        # g_yy component
    [0, 0, 0, a**2],        # g_zz component
]
```

6.2 Examples

Example 1: Minkowski Metric (Flat Spacetime)

The simplest spacetime, Minkowski metric, is obtained by setting $a(t) = 1$:

```
# 1. Coordinates
t, x, y, z = sp.symbols("t␣x␣y␣z")
coords = [t, x, y, z]
c = sp.Symbol("c", positive=True, real=True)

# 2. Minkowski metric
metric_arr = [
    [-c**2, 0, 0, 0],
    [0, 1, 0, 0],
    [0, 0, 1, 0],
    [0, 0, 0, 1],
]
```

This yields zero Christoffel symbols, vanishing Riemann and Ricci tensors, $R = 0$, and $G_{\mu\nu} = 0$, as expected for flat spacetime.

Example 2: Custom Spherically Symmetric Metric

For a general static, spherically symmetric metric of the form

$$ds^2 = -A(r)c^2dt^2 + B(r)dr^2 + r^2d\theta^2 + r^2\sin^2\theta d\phi^2, \tag{14}$$

the user defines:

```
# 1. Coordinates
t, r, theta, phi = sp.symbols("t␣r␣theta␣phi")
coords = [t, r, theta, phi]
coord_names = ['t', 'r', '\\theta', '\\phi']

# 2. Define metric functions
A = sp.Function("A")(r)
B = sp.Function("B")(r)
```

```
# 3. Custom metric
metric_arr = [
    [-A(r), 0, 0, 0],
    [0, B(r), 0, 0],
    [0, 0, r**2, 0],
    [0, 0, 0, r**2 * sp.sin(theta)**2],
]
```

The software then automatically computes all tensors in terms of $A(r)$, $B(r)$, and their derivatives. This framework can accommodate any metric expressible in symbolic form, including Kerr (rotating black hole), Reissner-Nordström (charged black hole), or Bianchi (anisotropic cosmological) metrics.

6.3 Important Considerations

When defining a custom metric, users should carefully attend to several details to ensure correct tensor calculations and LaTeX output.

First, **coordinate names** must be consistent between the coordinate list and the metric array indices. The software uses these names both for symbolic manipulation and for generating readable LaTeX output. Mismatches can lead to silent errors or incorrectly labeled tensor components.

Second, **LaTeX labels** in the `coord_names` list require proper escaping for special characters. For example, the angular coordinate θ must be entered as `'\\theta'` so that the LaTeX generator produces the correct math mode formatting. The same applies to ϕ (`'\\phi'`) and any other non-alphabetic symbols.

Third, the **metric signature** should be preserved according to the conventions of general relativity. The standard signature in most GR applications is $(-,+,+,+)$ (one timelike dimension and three spacelike dimensions). Altering this convention will propagate through all derived tensors, potentially leading to sign errors in physical quantities such as the Ricci scalar or Einstein tensor.

Fourth, **symbolic parameters** should be declared with appropriate mathematical assumptions using SymPy's symbol attributes. Specifying `positive=True` and `real=True` for physical quantities like mass M, gravitational constant G, speed of light c, and radial coordinate r helps SymPy simplify expressions more aggressively, eliminating spurious absolute values and piecewise conditions. For example:

```
M = sp.Symbol("M", positive=True, real=True)
r = sp.Symbol("r", positive=True, real=True)
```

This is particularly important for the Kretschmann scalar, where cancellations of terms like $(R_s - r)^2$ rely on SymPy's ability to assume $r > 0$.

Finally, users should verify that the **coordinate ordering** in the metric array matches the order of the coordinate list. The standard ordering in this software is (t, x, y, z) for Cartesian coordinates and (t, r, θ, ϕ) for spherical coordinates. The tensor computation routines assume this ordering throughout.

6.4 Extending to Other Spacetimes

The modular architecture of EinsteinKPy2Tex allows straightforward extension to any 4-dimensional metric beyond the two built-in examples. The user needs only to:

1. **Define the coordinate system**: Choose appropriate coordinate symbols (e.g., (t, x, y, z), (t, r, θ, ϕ), or (u, v, θ, ϕ) for null coordinates). The number of coordinates determines the spacetime dimension, which the software automatically propagates to all tensor calculations.

2. **Declare metric functions and parameters**: Use SymPy's `Function` class for position-dependent metric components (e.g., $A(r)$, $B(r)$, $\Psi(t, r)$) or `Symbol` for constants (e.g., mass

M, charge Q, cosmological constant Λ). The software will treat these symbolically, preserving them in the output expressions.

3. **Construct the metric tensor array**: Create a 4×4 (or $D \times D$ for other dimensions) list of lists, where diagonal entries represent the metric components along each coordinate direction, and off-diagonal entries represent cross-terms. The array should be symmetric, as the metric tensor is symmetric by definition:

```
metric_arr = [
    [g_tt, g_tr, 0, 0],
    [g_tr, g_rr, 0, 0],
    [0, 0, g_thth, 0],
    [0, 0, 0, g_phiph],
]
```

4. **Run the script** to obtain LaTeX output. The software automatically computes all derived quantities without further user intervention.

All subsequent tensor calculations—Christoffel symbols, Riemann curvature tensor, Ricci tensor and scalar, Einstein tensor, geodesic equations, and Kretschmann scalar—are handled automatically by the EinsteinPy backend [2] and the custom contraction routines described in Section **??**. This automation eliminates manual algebra errors and ensures consistency across all derived quantities.

7 Verification and Validation

We verify our outputs against known analytical results from the general relativity literature. These tests confirm the correctness of both the EinsteinPy backend and our custom contraction routines.

7.1 Schwarzschild Validation

For the Schwarzschild metric, which describes the spacetime around a non-rotating, uncharged black hole, we perform the following checks:

- **Ricci tensor**: The vacuum Einstein equations require $R_{\mu\nu} = 0$ outside the black hole. Our code returns zero for all components of the Ricci tensor, confirming the vacuum solution property.
- **Kretschmann scalar**: The fully contracted Riemann tensor product yields the well-known curvature invariant:
$$K = \frac{48G^2M^2}{c^4r^6} = \frac{12R_s^2}{r^6}, \tag{15}$$
where $R_s = 2GM/c^2$ is the Schwarzschild radius. This expression correctly diverges as $r \to 0$ (the physical singularity) while remaining finite at $r = R_s$ (the coordinate singularity at the event horizon).
- **Ricci squared and Gauss–Bonnet scalar**: The vacuum solution gives $R_{\mu\nu} = 0$, so $R_{\mu\nu}R^{\mu\nu} = 0$ and $\mathcal{R}_{\mathrm{GB}} = R_{\mu\nu\rho\sigma}R^{\mu\nu\rho\sigma} = K$. Our code confirms these identities.
- **Weyl tensor and Weyl squared**: For the Schwarzschild metric, the Weyl tensor is non-zero and its squared invariant $C_{\mu\nu\rho\sigma}C^{\mu\nu\rho\sigma}$ is computed by the code. The numerical value is not shown here, but the expression agrees with known results (e.g., $C_{\mu\nu\rho\sigma}C^{\mu\nu\rho\sigma} = \frac{48G^2M^2}{c^4r^6}$ for Schwarzschild).
- **Geodesic equations**: Our derived geodesic equations reproduce the standard orbital equations for massive and massless particles, including the perihelion precession of Mercury and light deflection by the Sun.

7.2 FLRW Validation

For the flat FLRW metric, which describes a homogeneous and isotropic expanding universe, we verify:

- **Ricci scalar**: The scalar curvature simplifies to

$$R = \frac{6}{a^2}\left(\frac{\ddot{a}}{a} + \frac{\dot{a}^2}{c^2}\right), \tag{16}$$

  where $a(t)$ is the scale factor and dots denote derivatives with respect to cosmic time t. This matches standard cosmology textbooks.

- **Einstein tensor components**: The time-time component yields the Friedmann equation:

$$G_{tt} = 3\frac{\dot{a}^2}{a^2} = \frac{8\pi G}{c^4}\rho c^2, \tag{17}$$

  correctly relating the expansion rate to the energy density ρ.

- **Kretschmann scalar**: Our code produces

$$K = \frac{12\left(a^2\ddot{a}^2 + \dot{a}^4\right)}{c^4 a^4}, \tag{18}$$

  which reduces to zero for Minkowski spacetime ($a =$ constant) as expected.

- **Weyl tensor and Weyl squared**: Since the FLRW metric is conformally flat, the Weyl tensor is identically zero. Our code returns zero for all components of $C^{\mu}_{\nu\rho\sigma}$ and for $C_{\mu\nu\rho\sigma}C^{\mu\nu\rho\sigma}$, confirming the property.

- **Ricci squared and Gauss–Bonnet scalar**: The code produces explicit expressions for $R_{\mu\nu}R^{\mu\nu}$ and $\mathcal{R}_{\mathrm{GB}}$ in terms of $a(t)$ and its derivatives. These are not shown here for brevity, but they have been verified to be consistent with the analytic formulas derived from the FLRW metric.

7.3 Cross-Validation

To ensure numerical stability, we also test the Schwarzschild metric in the weak-field limit ($r \gg R_s$), where it should approximate the Newtonian limit. The geodesic equations reduce to Newton's law of gravitation, and the Kretschmann scalar scales as r^{-6}, consistent with the inverse-square law of the gravitational field strength.

These validations confirm that `EinsteinKPy2Tex` produces correct results for both vacuum and cosmological spacetimes.

8 Educational Applications

The software is particularly well-suited for several educational and research contexts:

- **GR Coursework**: Students can modify metric components and immediately see how the Christoffel symbols, Riemann tensor, and geodesic equations change. This interactive exploration deepens understanding of curvature and its relationship to the metric.

- **Research Quick-Checks**: Researchers can rapidly verify metric properties, such as whether a proposed metric satisfies Einstein's equations or whether it contains singularities (via the Kretschmann scalar). This accelerates the iterative process of metric construction.

- **Publication Output**: The direct LaTeX export with non-zero components only allows authors to include tensor calculations directly in papers without manual typesetting. This reduces transcription errors and saves significant time.
- **Self-Study**: Graduate students and self-learners can visualize which tensor components are non-zero and how they depend on coordinates, building intuition for curved spacetimes.
- **Code Reuse**: The modular design allows users to extract specific calculation routines for integration into larger projects, such as numerical relativity codes or cosmological parameter estimation pipelines.

9 Comparison with Existing Tools

Several software packages exist for symbolic tensor calculations in general relativity. Among the most widely used are `GRTensor` (a Maple package) and `xAct` (a suite for Mathematica). Both are powerful and flexible, offering a wide range of tensor operations, including the computation of Christoffel symbols, Riemann, Ricci, and Weyl tensors, as well as geodesic equations and curvature invariants. However, they require a commercial license for the underlying computer algebra system (Maple or Mathematica), which may limit accessibility for some researchers and educational institutions. Furthermore, neither package provides direct, automatic LaTeX export of the computed components, let alone filtering of non-zero entries with proper index formatting. Users typically need to manually copy or write custom code to generate publication-ready output.

Base `EinsteinPy`, a Python library upon which this work is built, offers open-source symbolic tensor calculations for metrics defined by the user. It correctly computes the fundamental tensors (Christoffel, Riemann, Ricci, Einstein). However, it does not automatically derive geodesic equations, nor does it compute higher-order invariants such as the Kretschmann scalar, the Gauss–Bonnet scalar, or the Weyl squared invariant. It also lacks any built-in facility for LaTeX export or citation management.

`EinsteinKPy2Tex` (this work) fills these gaps while retaining the advantages of Python and the EinsteinPy backend. The software provides:

- **Complete tensor suite**: Christoffel symbols, Riemann, Ricci, Einstein, Weyl.
- **Geodesic equations**: Derived automatically from the Christoffel symbols.
- **All quadratic curvature invariants**: Kretschmann scalar (K), Ricci squared ($R_{\mu\nu}R^{\mu\nu}$), Gauss–Bonnet scalar ($\mathcal{R}_{\rm GB} = R^2 - 4R_{\mu\nu}R^{\mu\nu} + R_{\mu\nu\rho\sigma}R^{\mu\nu\rho\sigma}$), and Weyl squared ($C_{\mu\nu\rho\sigma}C^{\mu\nu\rho\sigma}$).
- **Direct LaTeX export**: Only non-zero components are exported, with proper spacing for multi-character indices (e.g., $\Gamma^{t}_{\phi t}$ instead of $\Gamma^{t}_{\phi t}$). The output is a complete, ready-to-compile LaTeX document.
- **Built-in citations**: The generated LaTeX file and the command-line interface include proper citations for Einstein (1915), the EinsteinPy library, and the software itself.
- **Open source and free**: The entire code is released under the MIT license, requires no commercial software, and is permanently archived on Zenodo.

Table 1 summarizes the feature differences between `EinsteinKPy2Tex`, `GRTensor`, `xAct`, and base `EinsteinPy`. The "K" in the name highlights the Kretschmann scalar, but the software goes far beyond that, offering a complete set of quadratic invariants. Notably, while `GRTensor` and `xAct` can in principle compute the Gauss–Bonnet scalar by manually contracting pre-computed tensors, they do not provide a single, built-in, pre-simplified function that outputs the final expression directly. `EinsteinKPy2Tex`, in contrast, exports the fully simplified symbolic form of $\mathcal{R}_{\rm GB}$ as a ready-to-use LaTeX formula. The same holds for the Weyl squared invariant. This

level of automation, combined with direct LaTeX export and open-source accessibility, makes `EinsteinKPy2Tex` uniquely suited for educators, students, and researchers who need rapid, accurate, publication-ready results.

Feature	**EinsteinKPy2Tex**	**GRTensor**	**xAct**	**EinsteinPy**
Open source	✓	×	✓	✓
Python-based	✓	×	×	✓
LaTeX export	✓	×	×	×
Geodesic equations	✓	✓	✓	×
Kretschmann scalar	✓	✓	✓	×
Gauss–Bonnet scalar	✓	[1]	[1]	×
Weyl tensor / Weyl squared	✓	✓	✓	×
Built-in citations	✓	×	×	×
Zero-cost	✓	×	✓	✓
Platform	Python	Maple	Mathematica	Python

[1] Can be constructed manually by the user from pre-computed tensors, but not available as a single, built-in, pre-simplified function.

Table 1: Comparison of `EinsteinKPy2Tex` with existing GR tensor calculation tools. The software uniquely provides a complete set of quadratic curvature invariants (Kretschmann, Ricci squared, Gauss–Bonnet, Weyl squared) together with LaTeX export, geodesic equations, and built-in citations, all under an open-source license.

10 Future Work

Several enhancements are planned for future releases of `EinsteinKPy2Tex`:

- **Kerr metric**: Support for rotating black holes, including the Kerr metric and the Kerr-Newman generalization with charge. This requires handling off-diagonal metric components ($g_{t\phi} \neq 0$) and more complex geodesic equations.
- **FLRW with curvature**: Extension to non-flat FLRW metrics with spatial curvature $k = \pm 1$ (closed and open universes). This involves additional metric components and modifications to the Kretschmann scalar.
- **Command-line metric definition**: A user-friendly interface allowing custom metric definitions without modifying the source code, perhaps via a configuration file or interactive prompts.
- **Jupyter notebook integration**: Interactive notebooks with live visualization of tensor components, geodesic trajectories, and curvature invariants as functions of coordinates.
- **Plotting capabilities**: Automatic generation of plots for the Kretschmann scalar as a function of radius, showing the singularity structure of black hole spacetimes.
- **Web-based interface**: A browser-based interface for non-Python users, enabling metric definition and tensor computation without local installation.

Long-term prospects

While `EinsteinKPy2Tex` currently focuses on local differential geometric quantities, future extensions could include:

- **Topological invariants**: Computation of the Euler characteristic $\chi(M)$ and Chern classes for compact spacetimes, using the Gauss-Bonnet theorem and characteristic class formulas. This would require integration over the manifold, not just local tensor evaluation.
- **Pontryagin classes**: Calculation of Pontryagin classes for gravitational instantons and other topologically non-trivial spacetimes in Euclidean quantum gravity.
- **AdS/CFT applications**: Computation of Weyl tensor invariants and conformal anomalies relevant to the AdS/CFT correspondence.

However, these topological extensions require global manifold information (compactness, boundaries, orientation) and integration routines that are beyond the current scope of `EinsteinKPy2Tex`. They remain an interesting direction for future research and collaboration.

11 Conclusions

`EinsteinKPy2Tex` provides a reliable, user-friendly bridge between symbolic GR calculations and publication-ready LaTeX. By automating tensor computations for the most important metrics in cosmology and black hole physics, it saves researchers and educators significant time while reducing errors. The latest version of the software furthermore includes the Ricci squared, Gauss–Bonnet, and Weyl squared invariants, as well as the full Weyl tensor, providing a comprehensive suite of quadratic curvature invariants. The built-in citation system ensures proper academic attribution, and the MIT license encourages wide adoption and contribution.

12 Data and Code Availability

The software is open-source under the MIT license and available at:
`https://github.com/lontelis/EinsteinKPy2Tex`

Acknowledgments

The author thanks the EinsteinPy development team for their excellent library, and DeepSeek AI for assistance with code optimization and documentation.

References

[1] Aaron Meurer, Christopher P. Smith, Mateusz Paprocki, Ondřej Čertík, Sergey B. Kirpichev, Matthew Rocklin, Amit Kumar, Sergiu Ivanov, Jason K. Moore, Sartaj Singh, Thilina Rathnayake, Sean Vig, Brian E. Granger, Richard P. Muller, Francesco Bonazzi, Harsh Gupta, Shivam Vats, Fredrik Johansson, Fabian Pedregosa, Matthew J. Curry, Andy R. Terrel, Štěpán Roučka, Ashutosh Saboo, Isuru Fernando, Sumith Kulal, Robert Cimrman, and Anthony Scopatz. SymPy: symbolic computing in python. *PeerJ Computer Science*, 3:e103, 2017.

[2] Shreyas Ribeiro, Aaryan Bapat, Ayush Tandon, and EinsteinPy Developers. EinsteinPy: A Python Package for General Relativity. *Journal of Open Source Software*, 5(51):2356, 2020.

[3] Antony Lewis, Anthony Challinor, and Anthony Lasenby. Efficient Computation of Cosmic Microwave Background Anisotropies in Closed Friedmann-Robertson-Walker Models. *The Astrophysical Journal*, 538(2):473–476, 2000.

[4] Diego Blas, Julien Lesgourgues, and Thomas Tram. The Cosmic Linear Anisotropy Solving System (CLASS) II: Approximation schemes. *Journal of Cosmology and Astroparticle Physics*, 2011(07):034, 2011.

[5] Julien Lesgourgues. The Cosmic Linear Anisotropy Solving System (CLASS) I: Overview. *arXiv e-prints*, 2011.

[6] Julien Lesgourgues. The Cosmic Linear Anisotropy Solving System (CLASS) III: Comparison with CAMB for LambdaCDM. *arXiv e-prints*, 2011.

[7] Uroš Seljak and Matias Zaldarriaga. A Line-of-Sight Integration Approach to Cosmic Microwave Background Anisotropies. *The Astrophysical Journal*, 469:437, 1996.

[8] Planck Collaboration, N. Aghanim, Y. Akrami, et al. Planck 2018 results. VI. Cosmological parameters. *Astronomy & Astrophysics*, 641:A6, 2020.

[9] Antony Lewis and Sarah Bridle. Cosmological parameters from CMB and other data: A Monte Carlo approach. *Physical Review D*, 66(10):103511, 2002.

[10] Benjamin Audren, Julien Lesgourgues, Karim Benabed, et al. Monte Python: simulating the Planck likelihood. *Journal of Cosmology and Astroparticle Physics*, 2013(02):001, 2013.

[11] A. Spurio Mancini, D. Piras, J. Alsing, et al. CosmoPower: Emulating cosmological power spectra for accelerated Bayesian inference from next-generation surveys. *Monthly Notices of the Royal Astronomical Society*, 511(2):1771–1788, 2022.

[12] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime F. del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585:357–362, 2020.